



Team: Big potato

Project topic: Stochastic Yield Analysis for Circuit Components

Bridging Ideal Circuit Theory with Volume Manufacturing

Course no. : EEE 4408
Course Name : Random Signals and Processes Lab

Team Members:

Name	ID
Irfan Hossain Rafi	230021331
Shoaib Mahbub	230021325
Rafin Islam	230021323

MOTIVATION & REAL-WORLD APPLICATION

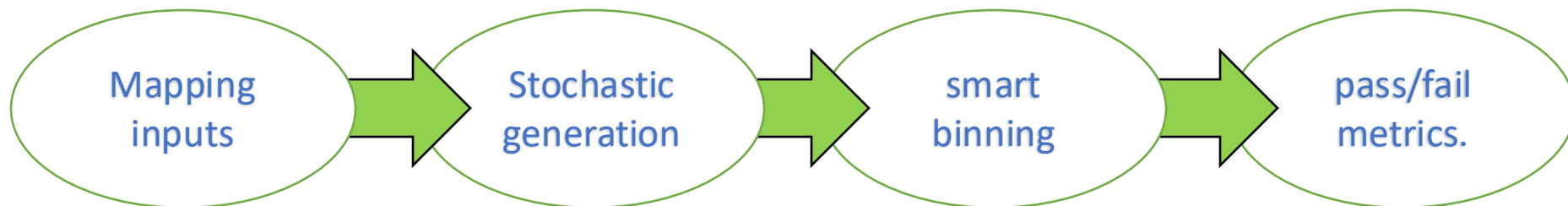
Core Problem: Creating a functional benchtop circuit is just 20% of the challenge, the remaining 80% involves guaranteeing that 1,000,000 units meet specifications on the assembly line.

Industry /Application	What Happens If Components Drift?	Real-World Impact
Noise-Canceling Headphones	Active filter cutoff frequency shifts by just 5%	Bass becomes muffled, and outside noise leaks in.
Smartwatch Health Sensors	Filter quality factor (Q) drifts out of spec	Sensor fails to clean up noise, giving inaccurate ECG/heart-rate readings.
High-Volume Smartphones	Factory buys expensive ultra-precise 0.1% components	Production cost explodes by millions of dollars for no extra feature.

Simulation Pipeline Architecture

4-Stage Modular Design:

1. **config_params**: Defines nominal values, tolerances, and $\pm 3\%$ yield boundaries.
2. **generate_components**: Creates 100,000-unit stochastic component populations.
3. **perform_combinatorics**: Executes random vs. smart bin-pairing assembly.
4. **calculate_cutoff**: Evaluates vectorized transfer functions (f_c and Q).



Module 1 (config_params)

- **Batch Setup:** $N = 100,000$ simulation runs with an anchored seed (`seed = 50`) for exact reproducibility.
- **Nominal Components:** $R_1, R_2 = 10\text{k}\Omega$, $C_1, C_2 = 10\text{nF}$.
- **Manufacturing Limits:** R Tolerance $= \pm 1\%$ (3σ level), C Tolerance $= \pm 6\%$
- **Dual-Target Specs:** Sets $\pm 3\%$ target specification windows for both cutoff frequency (f_c) and quality factor (Q).

```
%% 4. Target Design Specifications & Yield Limits
cfg.fc_target = 1 / (2 * pi * sqrt(cfg.R1_nom * cfg.R2_nom * cfg.C1_nom * cfg.C2_nom));
cfg.spec_tolerance = 0.03;           % +-3% frequency tolerance

cfg.fc_min = cfg.fc_target * (1 - cfg.spec_tolerance);
cfg.fc_max = cfg.fc_target * (1 + cfg.spec_tolerance);

%% Quality Factor (Q) Specifications
% Q = sqrt(R1 * R2 * C1 * C2) / (C2 * (R1 + R2))
cfg.Q_target = sqrt(cfg.R1_nom * cfg.R2_nom * cfg.C1_nom * cfg.C2_nom) / ...
    (cfg.C2_nom * (cfg.R1_nom + cfg.R2_nom));
cfg.Q_min = cfg.Q_target * (1 - cfg.spec_tolerance);
cfg.Q_max = cfg.Q_target * (1 + cfg.spec_tolerance);
```

Module 2 (generate_components)

Resistors (Gaussian / Normal Distribution):

- $\sigma R = \frac{R_{nom} \times R_{tol}}{R_{sigma_level}}$
- Models central-limit clustering in precision thin-film manufacturing.

Capacitors (Uniform Distribution):

- Models continuous flat tolerance spread typical of binned ceramic dielectrics.

```
%% 1. Resistors: Gaussian (Normal) Distribution
% Standard deviation derived so tolerance equals the specified sigma level
sigma_R1 = (cfg.R1_nom * cfg.R1_tol) / cfg.R_sigma_level;
sigma_R2 = (cfg.R2_nom * cfg.R2_tol) / cfg.R_sigma_level;

% Generate N continuous random variables following N(mu, sigma^2)
comp.R1 = cfg.R1_nom + sigma_R1 * randn(cfg.N, 1);
comp.R2 = cfg.R2_nom + sigma_R2 * randn(cfg.N, 1);
```

```
%% 2. Capacitors: Uniform Distribution
% Define lower and upper tolerance bounds [a, b]
C1_min = cfg.C1_nom * (1 - cfg.C1_tol);
C1_max = cfg.C1_nom * (1 + cfg.C1_tol);

C2_min = cfg.C2_nom * (1 - cfg.C2_tol);
C2_max = cfg.C2_nom * (1 + cfg.C2_tol);
```

Module 3 — perform_combinatorics (Core Innovation)

- **Random Assembly (`enable_smart_sorting = false`):**
 - Randomly shuffles components; unmanaged error stacking leads to high factory scrap.
- **Smart Bin-Pairing (`enable_smart_sorting = true`):**
 - Calculates effective metrics: $R_{eff} = \sqrt{R_1 R_2}$ and $C_{eff} = \sqrt{C_1 C_2}$.
 - Sorts R_{eff} ascending and C_{eff} descending.
 - **Mechanism:** High resistance cancels low capacitance, holding $R_{eff} \cdot C_{eff}$ close to constant to stabilize f_c near target.

```
if ~cfg.enable_smart_sorting
    assembled.R1 = comp.R1(randperm(N));
    assembled.R2 = comp.R2(randperm(N));
    assembled.C1 = comp.C1(randperm(N));
    assembled.C2 = comp.C2(randperm(N));
```

```
else
    R_eff = sqrt(comp.R1 .* comp.R2);
    C_eff = sqrt(comp.C1 .* comp.C2);

    [~, idx_R] = sort(R_eff, 'ascend');
    [~, idx_C] = sort(C_eff, 'descend');

    % Pair sorted indices so high R cancels
    assembled.R1 = comp.R1(idx_R);
    assembled.R2 = comp.R2(idx_R);
    assembled.C1 = comp.C1(idx_C);
    assembled.C2 = comp.C2(idx_C);
end
```

Module 4 — calculate_cutoff

- **Vectorized Transfer Functions:**

- $f_c = \frac{1}{2\pi\sqrt{R_1R_2C_1C_2}}, Q = \frac{\sqrt{R_1R_2C_1C_2}}{C_2(R_1+R_2)}$

- **Performance Optimization:** Pre-calculates shared numerator terms to run 100,000 iterations simultaneously without for loops.

```
num = sqrt(assembled.R1 .* assembled.R2 .* assembled.C1 .* assembled.C2);  
den = assembled.C2 .* (assembled.R1 + assembled.R2);  
  
fc = 1 ./ (2 * pi * num);  
Q = num ./ den;
```

Main Execution & Dual-Constraint Pass/Fail

- **Joint Specification Logic:**

`pass_mask = (fc >= fc_min & fc <= fc_max) & (Q >= Q_min & Q <= Q_max)`

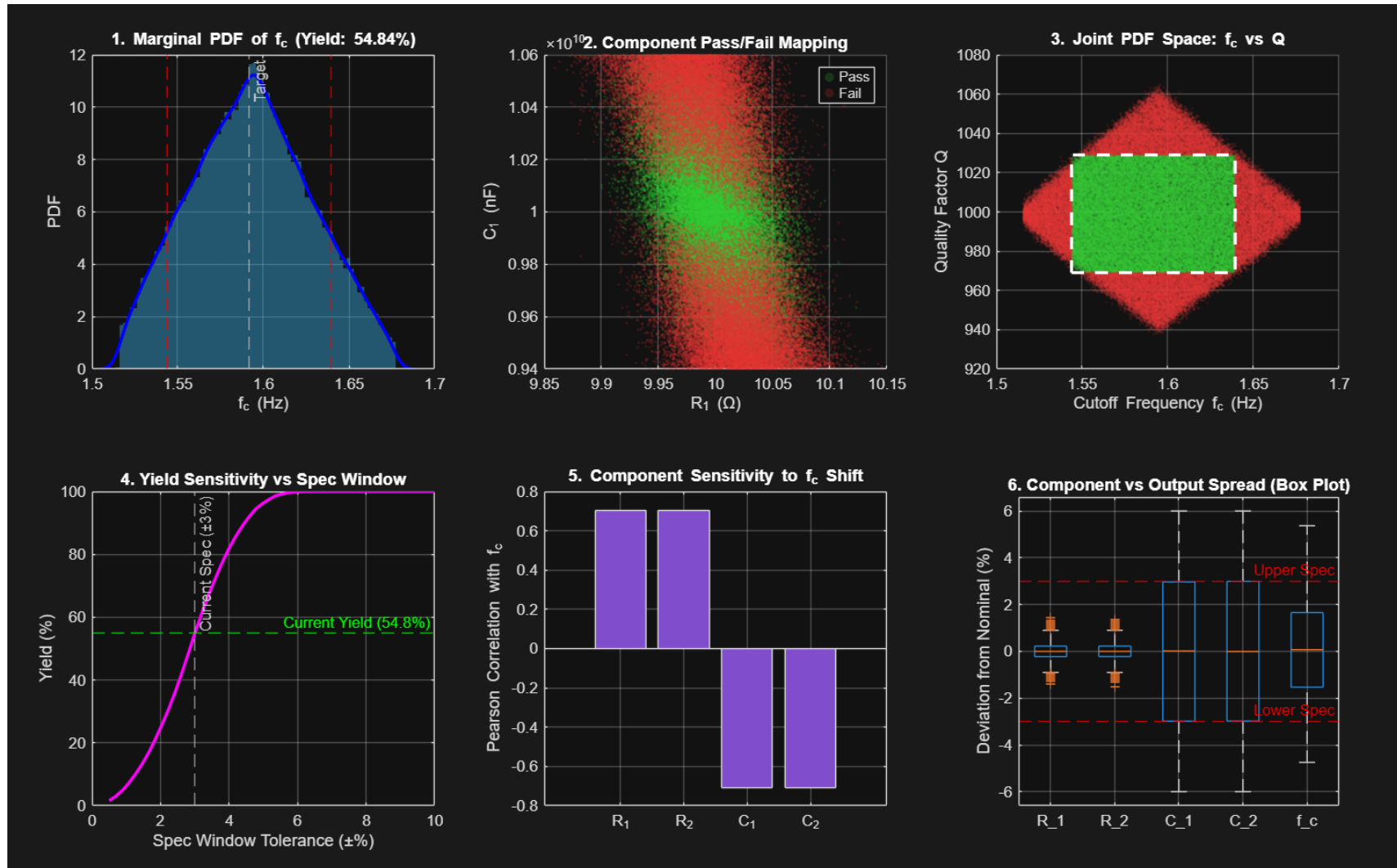
- **Engineering Rationale:** Evaluating fc alone isn't enough as Q drift causes unwanted passband peaking or signal attenuation. Both parameters must pass simultaneously.

```
%% 2. Joint Specification Yield Check (fc AND Q)
pass_mask = (fc >= cfg.fc_min & fc <= cfg.fc_max) & ...
            (Q >= cfg.Q_min & Q <= cfg.Q_max);
passed_units = sum(pass_mask);
yield_pct = (passed_units / cfg.N) * 100;
```

Dashboard Analytics Breakdown

- **Marginal PDF & Joint Space:** Displays distribution of f_c and plots f_c vs. Q against a rectangular spec box.
- **Pass/Fail Component Mapping:** Highlights passing (green) vs. failing (red) clusters on R_1 vs. C_1 axes.
- **Sensitivity Matrix:** Uses Pearson correlation to rank component impact on f_c drift, identifying C_1 and C_2 as primary variance drivers.

MATLAB Visual Dashboard



Business ROI & Financial Impact

- BOM Cost Savings:** Rescues yield using standard 6% capacitors instead of purchasing 5 times more expensive 1% parts.
- Scrap Reduction:** Prevents high parts-per-million (PPM) defect rates at the factory testing stage.
- Zero-Hardware-Cost Innovation:** Leverages software sorting at the SMT Pick-and-Place stage to improve quality at 0 additional component cost.